

Mathematical Foundation of Parallel Digital Resonance

A Flat-Space Coordinate Mapping of Grover's Algorithm

1. Vector Space & Coordinate Setup

Instead of an abstract Hilbert space of physical superpositions, the algorithm operates on a classical, flat vector space over a finite coordinate system.

Let the search space be an uncurved, orthogonal 1D coordinate space of size $N = 2^n$, where n is the number of bits in your index register ($n = 20$). Every possible state or logical proposition is assigned a discrete unit basis vector e_i .

$$\text{Basis Space: } \mathcal{V} = \{e_0, e_1, e_2, \dots, e_{N-1}\}$$

The status of the entire system at any instant is tracked by a state multivector ψ , which is a linear combination of these basis elements weighted by real-valued scalar amplitudes A_i :

$$\psi = \sum_{i=0}^{N-1} A_i e_i$$

The Orthogonality Condition

Because distinct logical propositions are mutually exclusive, their geometric inner product is identically zero:

$$e_i \cdot e_j = \delta_{ij} = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases}$$

2. The Initial Uniform Vector (DC Bias Setup)

To begin the process without a priori coordinate bias, energy is injected equally across all parallel lanes. In signal processing terms, this establishes a uniform DC offset across the array. The normalized uniform state vector s is defined as:

$$s = \frac{1}{\sqrt{N}} \sum_{i=0}^{N-1} e_i$$

For a 20-bit register space ($N = 2^{20} = 1,048,576$):

$$A_{\text{init}} = \frac{1}{\sqrt{1,048,576}} = \frac{1}{1024} \approx 0.0009765625$$

3. Geometric Algebra (GA) Bireflection Mechanics

Every iteration of the loop consists of two successive hyper-plane reflections. In Geometric Algebra, reflecting a state vector ψ across a hyperplane perpendicular to a unit vector u is executed via the geometric product:

$$\psi' = -u\psi u$$

Grover's algorithm is mathematically identical to a rigid **bireflection** (a product of two reflections) that rotates the state vector ψ inside a 2D plane spanned exclusively by the uniform vector s and the hidden target vector b .

Reflection 1: The Branchless Oracle

The oracle reflects the state vector ψ across the hyperplane orthogonal to the target proposition basis vector b . It evaluates a boolean verification function $f(i) \rightarrow \{0, 1\}$ branchlessly across all coordinates simultaneously:

$$\psi_1 = \psi - 2(\psi \cdot b)b \implies A'_i = (-1)^{f(i)} A_i$$

Reflection 2: The Diffusion Operator (DC Offset Removal)

The second step reflects the modified state vector ψ_1 back across the uniform state vector s :

$$\psi_2 = 2(s \cdot \psi_1)s - \psi_1$$

Because s represents the global uniform average, this operation translates exactly into a global subtraction of the DC component, or **Inversion About the Average**:

$$A''_i = 2\mu - A'_i$$

Where μ is the strict mathematical mean of the entire amplitude array:

$$\mu = \frac{1}{N} \sum_{j=0}^{N-1} A'_j$$

4. Multi-Target Numerical Convergence Scaling

If your 1D array contains M identical matching targets, the geometric rotation angle per loop iteration increases by a factor of $\sqrt{M}$. If you let the algorithm run for the standard single-target duration, the wave will over-modulate, destructively interfere, and alias back into the background noise floor.

The exact number of constant-bounded loop iterations R required to drive the target amplitude to its maximum constructive peak is governed by the following numerical analysis boundary:

$$R = \left\lfloor \frac{\pi}{4} \sqrt{\frac{N}{M}} \right\rfloor$$

Loop Iteration Mapping Table ($N = 2^{20}$)

Target Count (M)	Exact Closed-Form Formula	Constant Loop Boundary (R)
1 Target	$\lfloor \frac{\pi}{4} \sqrt{1,048,576} \rfloor$	804 iterations
2 Targets	$\lfloor \frac{\pi}{4} \sqrt{524,288} \rfloor$	568 iterations
4 Targets	$\lfloor \frac{\pi}{4} \sqrt{262,144} \rfloor$	402 iterations
8 Targets	$\lfloor \frac{\pi}{4} \sqrt{131,072} \rfloor$	284 iterations
16 Targets	$\lfloor \frac{\pi}{4} \sqrt{65,536} \rfloor$	201 iterations
32 Targets	$\lfloor \frac{\pi}{4} \sqrt{32,768} \rfloor$	142 iterations

5. Q15.16 Pure Integer Fixed-Point Conversion

To saturate the 512-bit integer vector ALU ports on AMD Zen 5 hardware and bypass floating-point latencies, the real-valued amplitudes are mapped to signed 32-bit integers using a **Q15.16 fixed-point scaling factor** ($2^{16} = 65,536$).

Fixed-Point Bit Allocation Profile

- **Bits [31 down to 16]:** Signed Integer Component (15 Bits + 1 Sign Bit)
- **Bits [15 down to 0]:** Fractional Component (16 Bits)

Pure Integer Mathematical Substitutions

- **Uniform State Initialization:**

$$A_{\text{fixed}} = \text{Integer} \left(\frac{1}{1024} \times 2^{16} \right) = \mathbf{64}$$

- **Branchless Sign Multiplication:**

$$\text{Multiplier} = 1 - 2 \cdot \text{Boolean'Pos}(\text{Match})$$

- **Divisionless Mean Extraction:** Instead of an expensive CPU division operation by $N = 2^{20}$, the summation of the long integers is scaled down using a bitwise right-shift primitive:

$$\mu_{\text{fixed}} = \text{Shift_Right} \left(\sum A_j, 20 \right)$$